How You Build the Queue Decides the Winner: Re-Evaluating Duplicate Pull Request Detection on Coding-Agent PRs

Harshul Jain, Saurabh Yergattikar, Tanya Sah
Independent Researchers

***Abstract*—**Coding agents open many pull requests (PRs) that repeat work already waiting for review. We ask how well duplicate-PR detectors work on these PRs when they are tested the way they are used: a new PR arrives, and the tool must find its duplicate among the PRs open in the repository at that moment. From the public AIDev dataset we build 1767 duplicate pairs (313 opened by different authors) and 2965 PRs with no duplicate, and we compare 66 strategies, from text matching and existing bots to embeddings, re-rankers and LLM judges. We report three findings. First, duplicates are costly: for cross-author pairs, 45% were flagged by a person more than an hour after the second PR opened, 17% more than a day later, and in 54% of cases someone had already commented on or reviewed the doomed PR. Second, small choices in how the open-PR queue is built change which method wins. When the queue also holds PRs closed in the previous seven days, our learned detector Jinx beats a classic research detector by +9.1 points of Recall@1. When the queue holds only open PRs, as a deployed tool sees it, the two are tied (0.927 vs 0.914), and a simple rule that checks for a shared issue or a cross-reference does as well (0.936). Third, the methods differ much more in false alarms than in recall: Jinx raises 1.8% false alarms, against 5.5% for the best existing bot, while an LLM judge on Jinx's top five lowers this to 0.9%. Duplicates that change different files remain hard for every method.

I. Introduction

AI coding agents such as Codex, Copilot, Devin, Cursor and Claude Code now open hundreds of thousands of pull requests (PRs) on GitHub [1]. Many are never merged, and duplicated work is a leading reason. In a hand-labelled sample of 600 failed agent PRs, 142 (23.7%) were duplicates of other PRs [2], and "resolved by another PR" is the most common reason why agent fix-PRs stay unmerged [3]; see also [4], [5].

Detecting duplicate PRs is an established research problem [6], [7], [8], [9], [10], [11], and several small bots now offer it on GitHub. Most detectors were scored on balanced sets of random PR pairs [7]. A deployed tool faces a different task: when a PR opens, it must rank the PRs that are open at that moment and decide whether to warn anyone. Work on duplicate bug reports showed that unrealistic set-ups can make methods look better than they are [12]. We find that the same is true for PRs, and that even a "realistic" set-up can mislead if the queue is not built exactly as the tool would see it.

We first built a live-queue benchmark whose queues also kept PRs closed in the previous seven days, and our learned detector, Jinx, won clearly. An independent review of our own draft asked whether this design favoured Jinx, because one of its signals is whether the older PR is still open. It did. With open-only queues the advantage disappears. We report both results because the contrast is itself a finding about how duplicate detectors should be evaluated.

Our contributions are:

- a measurement of what duplicate agent PRs cost maintainers: how long they go unnoticed and how much review they absorb;
- *LiveQueue*, a benchmark of 1767 duplicate pairs and 2965 PRs without a duplicate, built from 76,209 agent PRs, where each new PR comes with the PRs open at that time;
- a comparison of 66 strategies, including six open-source duplicate-PR bots and five LLM judges, with repository-level confidence intervals;
- evidence that two evaluation choices, random pairs and recently closed PRs in the queue, change which method appears best; and
- Jinx, an open-source tool, with all code, scores and results.

II. Research Questions

RQ1 What do duplicate agent PRs cost, and how long do they go unnoticed?
RQ2 How well do detectors find duplicates in the real open-PR queue, and how often do they raise false alarms?
RQ3 How much do evaluation choices change the conclusions?
RQ4 Which duplicates remain hard, and do LLMs help?

III. The LiveQueue Benchmark

A. Data source

We use AIDev v5 [1], a public dataset (CC BY 4.0) of PRs opened by coding agents on GitHub up to 31 March 2026. For each PR it provides the title and description, comments, reviews, linked issues, the changed files and the code changes.

B. Finding duplicate pairs

Maintainers and authors often say why they close a PR, for example "superseded by #123". We search PR comments for five phrases (*duplicate of*, *superseded by*, *in favor of*, *replaced by*, *same as*) followed by a PR number, as earlier datasets did [13], [11]. No person checked each pair, so we call these *silver* labels. In each pair, the newer PR is the *query* and the older PR is the duplicate the tool should find.

C. Building the queue

For each query, the queue holds the PRs in the same repository that were open when the query was created, capped at the 200 most recent, which is what the Jinx GitHub Action fetches. We keep a pair only if its duplicate is in this queue, which leaves 1767 pairs; the 200-PR cap removes 29 pairs (1.6%) whose duplicate was still open. To measure false alarms we add 2965 *no-duplicate queries*: merged PRs from the same repositories with no duplicate-like comment. A good tool should stay quiet on these. Link signals use only the query's own title and description, because the older PR's text in the dataset may have been edited after the duplicate was found.

In RQ3 we compare this with the queue design of our first draft, which also kept PRs that had closed less than seven days before the query ($n = 1867$ pairs, 342 cross-author).

D. Relations and kinds of duplicates

Of the 1767 pairs, 1454 are *self-replacements*, where the same author opened both PRs, and 313 are *cross-author* pairs (207 with the same agent, 106 with different agents). We focus on cross-author pairs, because a maintainer cannot spot them from the author's name. We also sort pairs by shared code: (A) *code twin*, at least half of the changed lines identical; (B) *partial overlap*; (C) *same files, different code*; and (D) *no shared files*, where the PRs reach the same goal through different files.

E. How good are the labels?

Some silver labels are not true duplicates; a small fix may be closed "in favor of" a large PR that includes it. We asked Claude Haiku 4.5 to read each pair and label it as duplicate, replacement, subsumed, related or unclear. In 200 random pairs, 79.0% (95% CI 73.5–84.5%) were duplicates or replacements, and 97.5% when subsumed pairs are included. For the 342 cross-author pairs the numbers are 84.8% and 96.2%. This is an LLM estimate, not a human check, and the same model also appears as a judge in RQ4; a human check of 100 pairs is planned.

IV. Strategies

We tested 66 strategies in seven families: simple rules (newest PR first, TF-IDF [6] and BM25 on raw text, template-stripped text or the diff); code overlap (shared files, overlapping line ranges, identical added lines); links (shared issue or a cross-reference); embeddings (MiniLM, BGE-small and BGE-base, UniXcoder, CodeBERT, Jina-code); existing bots (contrib-guard in two versions, Simili, PRSense, PRism, vector-triage, re-implemented with local models where they needed a paid service); learned combinations (logistic regression (LR) or gradient boosting over the INTRUDE signals plus a time feature [7], [8], which we call *INTRUDE + time*, over the Jinx signals, and over all signals); and re-rankers and LLM judges that re-score the top 5 or 10 candidates from Jinx (three cross-encoders, one fine-tuned BGE re-ranker, Qwen2.5 at 1.5B, 7B, Coder-7B and 14B, and Claude Haiku 4.5).

Jinx (Fig. 1) removes repeated agent boilerplate and compares the new PR with each open PR using text similarity (TF-IDF and a local MiniLM embedding), code overlap and context (shared issue, cross-reference, same author or agent, time gap). An LR model turns these signals into one score, and each signal that contributes becomes a short reason.

V. How We Measure

All learned strategies use five-fold cross-validation in which each fold holds out whole repositories. *Recall@1* is the share of queries whose true duplicate is ranked first, *Recall@5* the share where it is in the top five, and *MRR* rewards placing it near the top. For false alarms, each strategy gets a score threshold from its training folds (the median top score of cross-author duplicates), and we count how often the top candidate of a no-duplicate query passes it. Recall@1 is a ranking measure; at this threshold a tool actually *flags* only about half of the duplicates. All 95% confidence intervals use 2,000 bootstrap samples drawn by repository. Comparisons between two methods use paired differences on the same queries. Subgroup results are exploratory.

Before running the study we fixed one success test for Jinx: a clear, CI-backed gain over the best baseline on cross-author Recall@5. It fails in every setting we report.

VI. Results

A. RQ1: What do duplicates cost?

For each labelled pair we found the comment that marked the duplicate and measured the time from the second PR opening, the first moment a tool could warn, to that comment. For cross-author pairs the median was 30 minutes, but 45% took more than an hour, 17% more than a day and 6% more than a week (mean 46 hours). In 54% of cross-author cases a person had commented on or reviewed the PR that was later closed before the duplicate was noticed: 320 human comments and 304 reviews in total. The closed PRs added a median of 127 lines each, and across all relations 8.0 million added lines were thrown away.

If Jinx had commented when each new PR opened, it would have named the right duplicate in 46% of cross-author cases, 35% of those that people took more than a day to notice, at 1.8 false alarms per 100 PRs without a duplicate. This replay is an upper bound on attention saved, since a warning still has to be read.

B. RQ2: Detectors on the real queue

Table I gives the main results. On real-use queues, the strong methods are close: Jinx reaches 0.927, INTRUDE + time 0.914, contrib-guard 0.914, plain TF-IDF 0.911 and the link rule 0.936. Every Recall@1 difference between Jinx and these baselines has a confidence interval that includes zero, and so does Recall@5 (Jinx 0.990, INTRUDE + time 0.987).

The methods differ more in how often they cry wolf (Fig. 2). Jinx raises 1.8% false alarms. The best existing bot raises 5.5% (difference +3.7 points, CI +2.2 to +5.1) and TF-IDF 3.5% (+1.8, CI +0.9 to +2.8). INTRUDE + time (2.0%) and the link rule (1.2%) are not clearly different from Jinx.

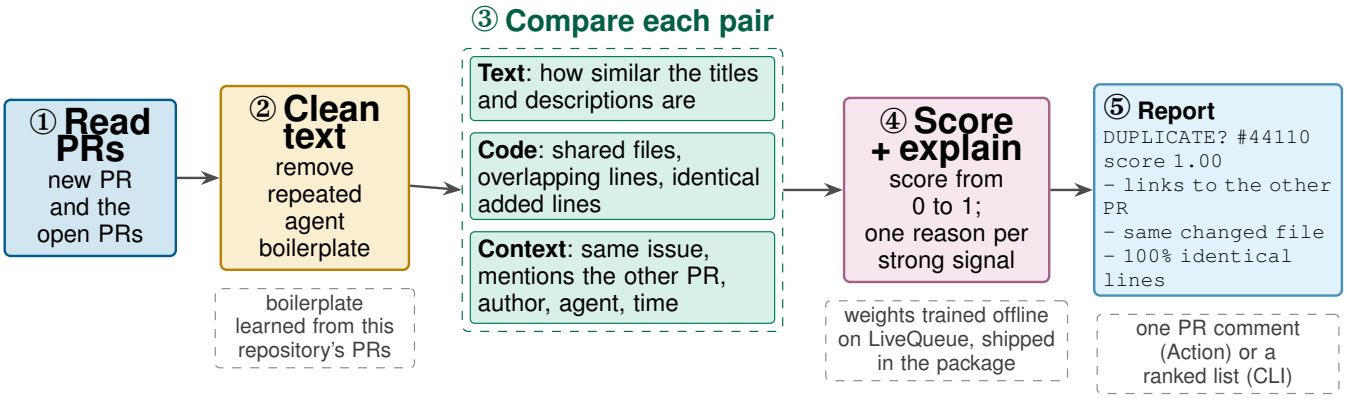


Fig. 1. How Jinx works. It compares a new PR with each open PR using text, code and context signals, combines them into one score, and explains each likely match in plain words.

TABLE I

CROSS-AUTHOR PAIRS ON REAL-USE QUEUES ($n = 313$); FALSE ALARMS ON 2965 PRS WITHOUT A DUPLICATE, WITH 95% CIS. UNDERLINED: BEST BASELINE. GREEN CELL AND †: SIGNIFICANTLY MORE FALSE ALARMS THAN JINX; ‡: SIGNIFICANTLY FEWER. THE LAST COLUMN IS JINX MINUS THE ROW IN RECALL@1 POINTS, WITH A PAIRED 95% CI.

Strategy	R@1 ↑	R@5 ↑	MRR ↑	False alarms % ↓	Jinx − row, R@1 (pts)
Our tool					
Jinx (ours, default)	0.927	0.990	0.954	1.8 [1.2, 2.5]	–
Jinx without MiniLM	0.917	0.990	0.949	1.9 [1.3, 2.7]	+1.0 [-0.9, +2.7]
Our tool with extra models added					
All signals, LR	0.927	0.990	0.955	1.6 [1.1, 2.2]	-0.0 [-2.0, +1.7]
Jinx + BGE rerank + Qwen-7B	0.936	0.981	0.958	1.4 [0.9, 2.0]	-1.0 [-2.6, +1.0]
Claude Haiku 4.5 + Jinx	0.936	0.990	0.961	0.9 [0.6, 1.3]‡	-1.0 [-2.5, +0.3]
Qwen2.5-14B + Jinx	0.917	0.990	0.949	1.3 [0.8, 2.0]‡	+1.0 [-0.0, +2.0]
Claude Haiku 4.5 alone on Jinx top-5	0.923	0.990	0.956	1.8 [1.1, 2.3]	+0.3 [-2.2, +2.6]
Baselines: link rule, classic detector, existing bot, embedding, text					
Link rule (shared issue or #ref)	0.936	0.978	0.957	1.2 [0.7, 1.7]	-1.0 [-4.3, +2.7]
INTRUDE + time [7], [8]	0.914	0.987	0.945	2.0 [1.3, 3.0]	+1.3 [-0.7, +3.1]
contrib-guard (bot)	0.914	0.981	0.947	5.5 [4.0, 7.0]†	+1.3 [-2.7, +4.7]
TF-IDF [6]	0.911	0.981	0.945	3.5 [2.5, 4.8]†	+1.6 [-2.2, +4.9]
BGE-small embedding	0.907	0.981	0.944	2.3 [1.5, 3.3]	+1.9 [-1.1, +4.6]

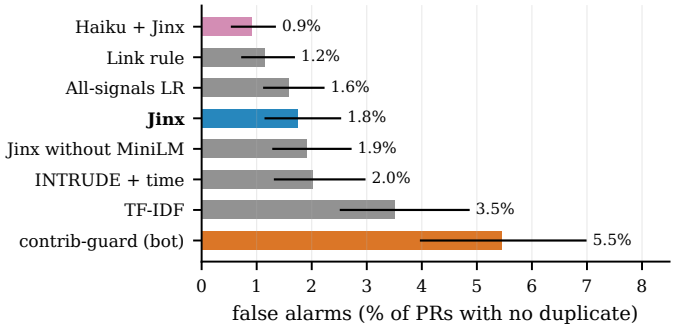


Fig. 2. False alarms on PRs with no duplicate, with 95% CIs. Each method uses the threshold at which it flags about half of the cross-author duplicates.

C. RQ3: Evaluation choices change the winner

Fig. 3 compares the two queue designs. With PRs closed in the previous seven days in the queue, Jinx reaches 0.901, INTRUDE + time 0.810, contrib-guard 0.778 and TF-IDF 0.757; Jinx's lead over INTRUDE + time is +9.1 points (CI +5.0 to +13.1). Closed PRs are always wrong answers, and Jinx has a signal for "the older PR is still open". Removing that signal drops Jinx to 0.839; giving it to INTRUDE + time lifts that method to 0.892. Most of the gain came from the

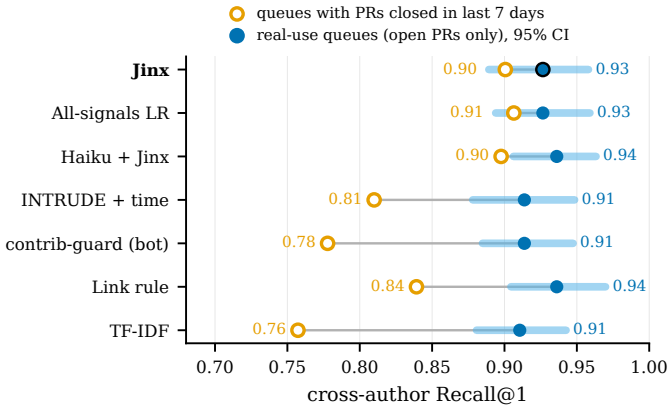


Fig. 3. Cross-author Recall@1 under two queue designs. Orange: queues that also hold PRs closed in the previous 7 days. Blue: real-use queues with open PRs only, with 95% CIs. The large gaps between methods in the first design vanish in the second.

queue design, not from better duplicate detection.

Random-pair scoring hides differences in the other direction (Fig. 4). The two scorings agree on the broad order (Kendall $\tau = 0.68$), mostly because weak methods are weak under both. Among strong methods, random-pair AUC is close to 1 and says little about which method ranks the right PR first.

D. RQ4: Hard duplicates and LLMs

Code twins are easy: every combined method ranks them first at least 0.95 of the time (Fig. 5). Duplicates that share no files (class D, $n = 25$) are harder: Jinx 0.720, INTRUDE + time 0.680, Haiku + Jinx 0.800. The link rule does well on these, which suggests that many intent-level duplicates are found through a shared issue. On the 118 cross-author pairs with no link at all, Jinx reaches 0.839 and INTRUDE + time 0.805.

LLM judges do not rank more duplicates first: Claude Haiku 4.5 averaged with Jinx reaches 0.936, within noise of Jinx, and the Qwen judges add nothing. They help in a different way. Averaging Haiku with Jinx lowers false alarms to 0.9% (CI of the difference -1.4 to -0.3 points), and at a 1% false-alarm

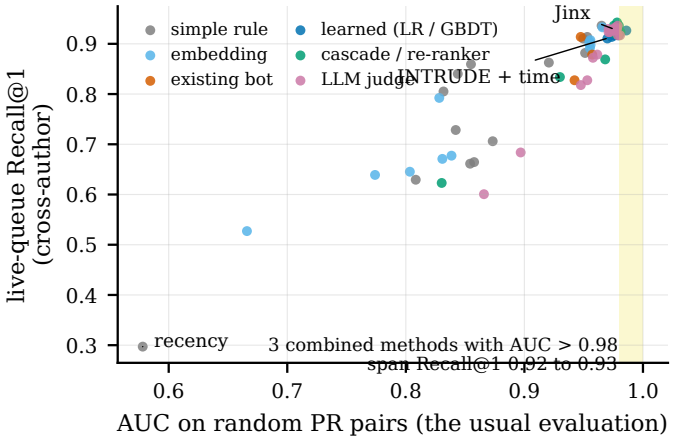


Fig. 4. Each dot is one strategy: AUC on random PR pairs (x) against live-queue Recall@1 (y).

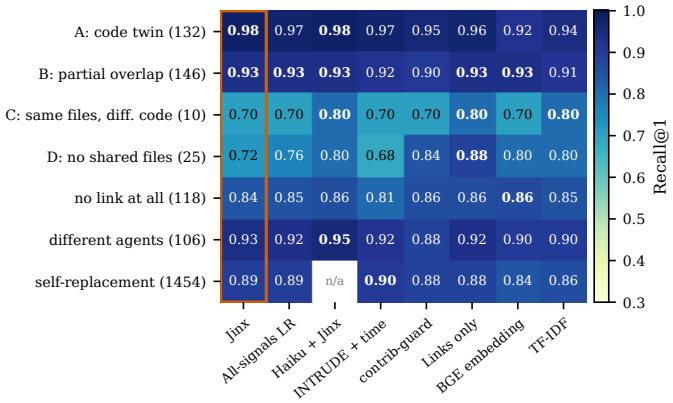


Fig. 5. Recall@1 by kind of duplicate (number of pairs in brackets), cross-author unless noted. Best value per row in bold; Jinx outlined. Small groups have wide uncertainty.

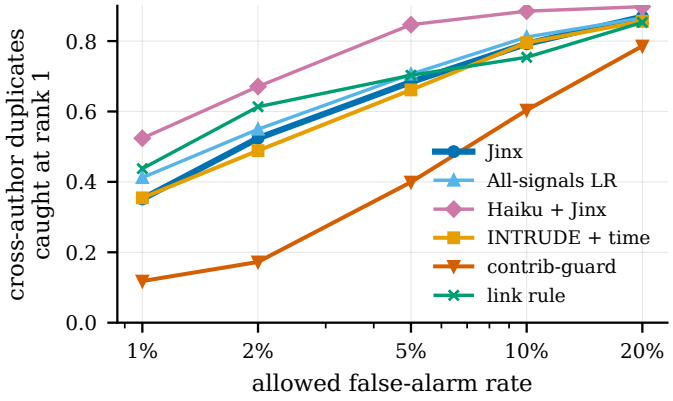


Fig. 6. Share of cross-author duplicates ranked first and flagged, for a fixed false-alarm budget on PRs without a duplicate.

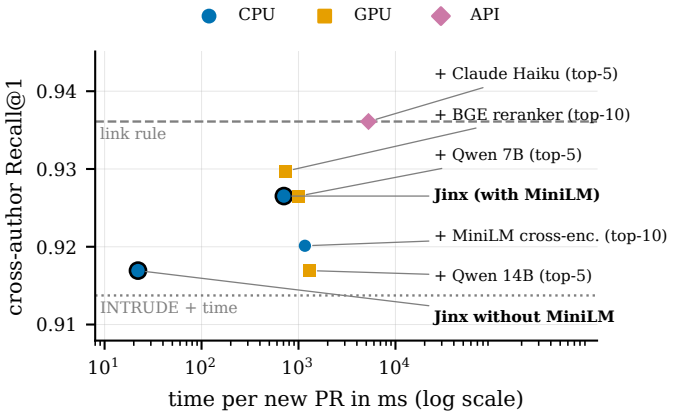


Fig. 7. Accuracy against time per new PR (median). Jinx times are end to end for the installed package on 4 CPU threads; added models are measured on top of Jinx with MiniLM.

budget it catches 52% of cross-author duplicates against 35% for Jinx and 44% for the link rule (Fig. 6). This costs an API call per candidate, and our run cost $7.25.

Fig. 7 shows cost. Without MiniLM, Jinx needs 22 ms per PR at the median (198 ms at the 90th percentile); with MiniLM, 0.7 s (3.7 s). Re-rankers and LLM judges add more time without a clear gain in Recall@1.

VII. Threats to Validity

Labels. Our labels come from comments, so unmarked duplicates are missed and some no-duplicate queries may hide real duplicates. The LLM audit estimates label quality but is not a human check, and on the 264 cross-author pairs it judged valid the comparison with INTRUDE + time does not change (Recall@1 difference +0.8 points, CI -1.3 to +2.7).

Links. People who close a duplicate often link the other PR, which makes link signals look strong. We count only links in the new PR's own text and report pairs with no link.

Queue design. The seven-day window was our own earlier choice and was stated in our data card, but none of us saw its effect until an independent review. Other choices, such as the 200-PR cap, may matter too.

Generality. We use one dataset and only agent PRs, so we cannot compare agent and human duplicates. Some bots were re-implemented and may behave differently from the deployed versions.

VIII. Related Work

Li et al. detected duplicate PRs with TF-IDF and ranked earlier PRs for each new PR [6], and released DupPR [13]. INTRUDE added code and issue signals [7], and later work added time [8], graph embeddings [9], LLM summaries [10] and issue-to-PR links [11]; see the review in [14] and studies of competing PRs [15], [16]. These works studied PRs by people and did not test how queue construction affects results. Studies of failed agent PRs list duplicates among the main causes [2], [3], [4], [5]; others study conflicts between concurrent agent PRs [17], coordination before a PR exists [18] and cross-references between PRs [19]. For duplicate bug reports, realistic evaluation shrank reported gains [12], and CUPID combines an LLM with retrieval [20].

IX. Conclusion

Duplicate agent PRs cost maintainers real attention, and a warning when the PR opens could save much of it. But which detector looks best depends on how the evaluation is built. Random pairs make strong methods look alike, and keeping recently closed PRs in the queue rewarded a signal that a deployed tool cannot use. On the real open-PR queue, a learned combination, a classic detector, plain TF-IDF and a simple link rule rank duplicates about equally well; they differ mainly in false alarms, where an LLM judge helps. We recommend that duplicate-PR studies build queues exactly as the tool will see them, report false alarms with confidence intervals, and report a simple link rule as a baseline.

Data Availability

The benchmark, code, raw scores, logs and the Jinx tool will be released under open licences with the camera-ready version.

References

[1] H. Li *et al.*, "AIDev: Studying AI coding agents on GitHub," 2026.

[2] R. Ehsani *et al.*, "Where do AI coding agents fail? an empirical study of failed agentic pull requests in GitHub," 2026, mSR 2026.

[3] K. Alam, S. Mondal, and B. Roy, "Why are AI agent involved pull requests (fix-related) remain unmerged?" arXiv:2602.00164, 2026.

[4] Nakashima *et al.*, "Why agentic-PRs get rejected: A comparative study of coding agents," 2026.

[5] M. Abujadallah, A. Arabat, and M. Sayagh, "Understanding the rejection of fixes generated by agentic pull requests: Insights from the AIDev dataset," arXiv:2606.13468, 2026.

[6] Z. Li, G. Yin, Y. Yu, T. Wang, and H. Wang, "Detecting duplicate pull-requests in GitHub," in *Proc. Internetware*, 2017.

[7] L. Ren, S. Zhou, C. Kästner, and A. Wąsowski, "Identifying redundancies in fork-based development," in *Proc. SANER*, 2019.

[8] Q. Wang, B. Xu, X. Xia, T. Wang, and S. Li, "Duplicate pull request detection: When time matters," in *Proc. Internetware*, 2019.

[9] J. Jiang, Q. Li, J. Guo, Z. Li, L. Zhang, and G. Li, "DupHunter: Detecting duplicate pull requests in fork-based development," *IEEE Trans. Softw. Eng.*, 2023.

[10] Z. Liao, Y. Liu, Y. Lan, and Y. Sun, "DupLLM: Duplicate pull requests detection based on large language model," in *Proc. APSEC*, 2024.

[11] R. Galbraith and D. Greer, "Detecting duplicate effort in GitHub contributions," in *Proc. ENASE*, 2025.

[12] T. Zhang, D. Han, V. Vinayakarao, I. C. Irsan, B. Xu, F. Thung, D. Lo, and L. Jiang, "Duplicate bug report detection: How far are we?" *ACM Trans. Softw. Eng. Methodol.*, 2023.

[13] Y. Yu, Z. Li, G. Yin, T. Wang, and H. Wang, "A dataset of duplicate pull-requests in GitHub," in *Proc. MSR*, 2018.

[14] B. Chekaya, Garrouch, and Omri, "Duplicate pull requests in code management platforms: A systematic literature review," Research Square preprint, 2026.

[15] X. Zhang, Y. Chen, Y. Gu, W. Zou, X. Xie, and X. Jia, "How do multiple pull requests change the same code: A study of competing pull requests in GitHub," in *Proc. ICSME*, 2018.

[16] Z. Li, Y. Yu, M. Zhou, T. Wang, and G. Yin, "Redundancy, context and preference: An empirical study of duplicate pull requests in OSS projects," *IEEE Trans. Softw. Eng.*, 2022.

[17] G. Xu, A. Subramanian, and N. Karthik, "AI agent pull requests on GitHub: Frequency, structure and merge conflict rates," arXiv:2607.04697, 2026.

[18] D. Sarkar, "Before the pull request: Mining multi-agent coordination," arXiv:2606.19616, 2026.

[19] Khemissi *et al.*, "Humans integrate, agents fix: How agent-authored pull requests are referenced in practice," 2026.

[20] T. Zhang, I. C. Irsan, F. Thung, and D. Lo, "CUPID: Leveraging ChatGPT for more accurate duplicate bug report detection," arXiv:2308.10022, 2023.